

武汉理工大学

企业级应用软件开发与开发 期末大作业

课程名称	企业级应用软件开发与开发
项目名称	海军标准智能体
方 向	Agentic AI 原生开发
学 号	2025302968
姓 名	程佳豪
专 业	软件工程
指导教师	戚欣
提交日期	2026 年 6 月 22 日

2025-2026 学年第 2 学期

目录

一、选题背景与设计思想	3
1.1 问题定义	3
1.2 现有方案不足	3
1.3 项目价值	3
1.4 技术路线	3
二、Specs 规格文档 (SDD 核心)	4
2.1 Product Spec (产品规格)	4
2.1.1 用户故事	4
2.1.2 功能需求 (FR)	4
2.1.3 非功能需求 (NFR)	5
2.2 Architecture Spec (架构规格)	5
2.3 API Spec (接口规格)	5
三、系统架构与设计	6
3.1 系统分层架构	6
3.2 Agent 交互流程	6
3.3 数据流设计	7
文档摄取流	7
检索增强生成流	7
3.4 核心设计决策	7
四、关键实现与代码展示	7
4.1 混合检索核心实现	7
4.2 LLM SSE 流式调用	8
4.3 提示词模板示例	8
4.4 Trae CN AI IDE 使用	9
4.5 安全措施	9
五、测试与评估	9
5.1 功能测试	9
5.2 Agent 行为评估	10
5.3 Demo 截图	10
六、系统升级与扩展	11
6.1 可扩展架构设计	11
6.2 下一阶段计划 (v2.1 → v3.0)	11
6.3 AI 能力演进路径	11
七、课程总结	12
7.1 个人收获	12
7.2 工程思维转变	12
7.3 对课程的建议	12

一、选题背景与设计思想

1.1 问题定义

海军标准文档（含国标 GB、国军标 GJB、海军标准 HJB/CB 等）是海军装备研制、试验、采购、使用和维修保障的重要技术依据。当前标准文档管理面临以下突出问题：

标准数量庞大：涉及多个层级和类别，传统关键词检索效率低下，难以精确定位条款；

条款引用不规范：人工查阅难以精确引用到具体标准编号和条款号，导致来源模糊；

合规校验依赖人工：制度/方案是否符合现行标准，缺乏自动化逐条校验工具；

标准更新滞后感知：新旧标准更替时，无法快速识别内容差距和修订需求；

文本质量参差不齐：海军公文中的口语化表达、术语不一致、格式不规范等问题普遍存在。

1.2 现有方案不足

传统的标准文档管理方案主要依赖全文检索（如 Elasticsearch）或关系型数据库的关键词匹配，存在以下局限：

关键词检索无法理解语义：用户输入"舰艇消防系统设计要求"，关键词匹配无法关联到"船舶灭火装置技术条件"等语义相近的标准条款；

缺乏智能推理能力：传统检索工具只能返回文档列表，无法基于检索结果进行综合分析和合规判断；

更新维护成本高：标准体系频繁更新，人工维护知识库的工作量巨大且易出错。

1.3 项目价值

本项目基于检索增强生成（RAG）技术，构建海军标准文档私有化智能问答系统，核心价值：

1. 精准检索：混合语义+关键词双路检索，RRF 融合排序，状态优先级加权，确保现行标准优先呈现；
2. 严格溯源：每条回答强制标注引用来源（标准编号+条款），区分强制执行/推荐/指导性条款；
3. 多功能覆盖：智能问答、文本优化、标准 GAP 分析、合规自查、文档管理五大功能；
4. 完全私有化：支持 Ollama 本地模型+离线嵌入，敏感标准数据不外传；
5. 开箱即用：Docker Compose 一键部署，降低运维门槛。

1.4 技术路线

本项目采用"RAG Pipeline + Agent 编排"技术路线：

文档处理层：PDF/DOCX/TXT 解析 → OCR 识别扫描版 → 文本清洗去噪 → 元数据提取 → 章节级分块（800 字+100 重叠）→ BGE 向量化（768 维）→ pgvector HNSW 索引入库。

检索增强层：用户问题向量化（LRU 缓存）→ 语义+关键词双路检索 → RRF 融合 → 状态优先级加权 → Trigram Jaccard 去重 → 上下文拼接。

生成响应层：系统提示词注入 → 检索结果注入 → LLM SSE 流式生成 → 前端 Markdown 实时渲染。

核心技术要素覆盖（≥4 项）：

SDD 规格驱动开发

工具使用 / Function Calling（LLM 调用+OCR+向量检索+数据库操作）

记忆机制（pgvector 向量数据库 + 多轮对话窗口）

状态管理与多步骤推理（RAG Pipeline: 检索→重排序→上下文增强→生成）

可观测性（日志轮转+分级+嵌入缓存命中率统计）

二、Specs 规格文档（SDD 核心）

2.1 Product Spec（产品规格）

2.1.1 用户故事

角色	需求
标准审查员	输入制度文本，系统自动检索相关标准并逐条校验合规性，输出三态合规报告
公文撰写者	粘贴海军公文，系统按文书规范自动优化口语化表达、统一术语、规整格式
标准管理员	上传新标准 PDF，系统自动解析、清洗、分块、向量化入库
技术决策者	输入技术方案，系统检索关联标准，输出五维度 GAP 分析报告

2.1.2 功能需求（FR）

编号	功能	说明
FR-01	智能问答	自然语言输入，检索相关标准条款，生成带来源标注的回答，SSE 流式输出
FR-02	文本优化	3 种风格 × 3 级强度，去口语化、统一术语、规整格式，显示变更统计
FR-03	标准 GAP 分析	5 维度诊断（完整性/对标性/时效性/可操作性/合规性），识别内容缺口
FR-04	合规自查	对照现行标准逐条校验，输出

		三态报告（符合/不符合/部分符合）
FR-05	文档管理	PDF/DOCX/TXT 上传，自动解析、OCR、清洗、元数据提取、分块、向量化
FR-06	SSE 流式响应	所有生成类 API 均支持 Server-Sent Events 流式推送

2.1.3 非功能需求（NFR）

NFR-01: 响应延迟 < 5s（不含 LLM 生成时间），嵌入模型本地 CPU 推理 < 50ms/条

NFR-02: 支持至少 3 种 LLM 后端切换（DeepSeek / 百炼 / Ollama），通过环境变量零代码切换

NFR-03: 敏感数据本地化，支持 OFFLINE_MODE 完全离线运行

NFR-04: 向量检索 HNSW 索引，10 万级文档库 < 10ms 返回

NFR-05: 前端零框架依赖，纯原生 HTML/CSS/JS，加载 < 1s

2.2 Architecture Spec（架构规格）

系统采用五层分层架构设计：

层级	组件	职责
前端层	原生 HTML/CSS/JS	SSE 流式消费、Markdown 渲染、多轮对话
API 网关层	FastAPI	RESTful 端点 + SSE 流式端点
RAG 引擎层	retrieval/ + rag/	检索增强 + 业务逻辑 + 提示词引擎
模型层	llm/ + embeddings/	LLM 客户端 + 嵌入模型 + OCR
存储层	PostgreSQL + pgvector	HNSW 向量检索 + 全文搜索 + 文件存储

核心设计原则：关注点分离（每层独立部署和扩展）、接口标准化（OpenAI 兼容协议，LLM 后端可替换）、数据本地化（敏感标准文档不出本地环境）、流式优先（SSE 流式响应，降低用户等待感知）。

详细架构图、Agent 交互流程、数据流设计见第三章。

2.3 API Spec（接口规格）

端点	功能	关键参数
POST /api/chat	智能问答（SSE）	messages, model, temperature
POST /api/optimize	文本优化（SSE）	text, style, intensity,

		dimensions
POST /api/gap-text	GAP 分析 (SSE)	text, domain, mode
POST /api/gap-analysis	标准 GAP 诊断	document_id, domain
POST /api/compliance	合规自查 (SSE)	text
POST /api/compare	标准对比 (SSE)	text1, text2, mode
POST /api/ingest	文档入库	file (multipart)
GET /api/documents	文档列表	page, page_size, status
DELETE /api/documents/{id}	删除文档	id

三、系统架构与设计

3.1 系统分层架构

系统采用五层架构设计（自上而下）：

前端层：原生 HTML/CSS/JS 实现，豆包/DeepSeek 风格极简界面。核心功能包括 SSE 流式消费、Markdown 实时渲染、多轮对话管理、文件上传。零框架依赖，单页应用。

API 网关层：FastAPI 异步框架，提供 RESTful 端点（文档 CRUD）和 SSE 流式端点（chat/optimize/gap/compliance/compare）。CORS 中间件 + StaticFiles 挂载。

RAG 引擎层：核心业务逻辑。检索模块（hybrid_search 语义+关键词双路检索 → RRF 融合 → 状态优先级加权 → Trigram 去重 → 多维筛选）；生成模块（compliance 合规校验 → gap_analyzer 五维 GAP 分析 → optimizer 文本优化 V2）；提示词引擎（14 个结构化模板，覆盖问答/摘要/优化/分析/合规/诊断/对比/修订场景）。

模型层：LLM 客户端（OpenAI 兼容协议，支持 DeepSeek/百炼/Ollama 切换，同步+SSE 流式）；嵌入模型（BAAI/bge-base-zh-v1.5 768 维，SentenceTransformer 本地 CPU 推理，@lru_cache 查询缓存）；OCR 引擎（Tesseract+中文语言包，扫描版 PDF 自动识别）。

存储层：PostgreSQL + pgvector（HNSW 索引，cosine 距离算子）；全文搜索（tsvector/tsquery）；文件系统（uploads/models/logs/tessdata）。

3.2 Agent 交互流程

智能问答完整交互流程：

- Step 1 — 用户输入问题，前端通过 EventSource 发起 SSE 连接 POST /api/chat;
- Step 2 — API 层构建消息上下文，注入 RAG_QA_SYSTEM_PROMPT 系统提示词;
- Step 3 — 调用 embed_query(问题) 获取 768 维查询向量（LRU 缓存命中率 >80%）;
- Step 4 — 并行执行语义检索(cosine <=>)和关键词检索(ts_rank)双路召回;
- Step 5 — RRF(K=60)融合双路结果，应用状态优先级权重(现行×1.0/修订×0.8/废止×0.5);
- Step 6 — Trigram Jaccard 去重(阈值 0.6)，合并相邻 chunk;

Step 7 — 拼接检索结果到 messages 上下文，注入领域策略和回答格式约束；
Step 8 — LLM SSE 流式生成，前端逐 token 实时渲染 Markdown，引用来源高亮显示。

3.3 数据流设计

文档摄取流

PDF/DOCX/TXT 文件 → 解析器判定类型 → [扫描版 PDF → OCR 识别] → 文本清洗(去噪/去页眉页脚/全角转换) → 元数据提取(标准编号/标题/状态/日期/领域) → 章节分块(800 字+100 重叠) → BGE 批量向量化(768 维) → 写入 document 表 + chunks 表 + pgvector 索引。

检索增强生成流

用户问题 → 查询向量化(LRU 缓存) → [语义检索 + 关键词检索] → RRF 融合 (K=60) → 状态优先级加权 → Trigram 去重(0.6) → 维度筛选(领域/年份/状态) → 上下文拼接 → 系统提示词注入 → LLM 生成 → SSE 流式响应。

3.4 核心设计决策

决策	方案	理由
混合检索权重	语义 0.7 + 关键词 0.3	军事标准术语密集、编号规范，需保留精确编号匹配
排序策略	RRF 融合	避免语义分数和关键词分数量纲不一致
状态处理	降权而非过滤	废止标准仍有历史参考价值
去重方式	Trigram Jaccard (0.6)	比整句哈希更鲁棒，允许少量文本差异
查询缓存	LRU maxsize=512	军事场景重复查询率高，命中率 >80%
分块策略	章节边界 + 800 字 + 100 重叠	标准文档天然按章节组织，边界即知识单元

四、关键实现与代码展示

4.1 混合检索核心实现

以下代码展示检索模块的核心——Semantic+Keyword 双路检索与 RRF 融合的实现逻辑（src/retrieval/hybrid_search.py）：

```
def hybrid_search(query: str, top_k: int = 10, **filters):  
    """语义+关键词混合检索，RRF 融合排序"""  
    # 1. 查询向量化（LRU 缓存）  
    query_vec = embed_query(query)
```

```

# 2. 双路并行检索
semantic_results = vector_search(query_vec, top_k=top_k * 2)
keyword_results = keyword_search(query, top_k=top_k * 2)

# 3. RRF 融合 (k=60)
fused = rrf_fusion([semantic_results, keyword_results], k=60)

# 4. 状态优先级加权 + 去重
fused = apply_status_priority(fused)
fused = dedup_by_similarity(fused, threshold=0.6)

# 5. 返回 Top-K
return fused[:top_k]

```

4.2 LLM SSE 流式调用

LLM 客户端封装 OpenAI 兼容协议，支持同步和 SSE 流式两种模式（src/llm/client.py）：

```

def chat_stream(messages, model=None, temperature=None, max_tokens=None):
    """SSE 流式对话 — 兼容 OpenAI 协议"""
    client = OpenAI(api_key=LLM_API_KEY, base_url=LLM_BASE_URL)
    response = client.chat.completions.create(
        model=model or LLM_MODEL,
        messages=messages,
        temperature=temperature or LLM_TEMPERATURE,
        stream=True,
        timeout=httpx.Timeout(60.0, connect=10.0),
    )
    for chunk in response:
        content = chunk.choices[0].delta.content
        if content:
            yield content

```

关键设计：通过 LLM_BASE_URL 环境变量，零代码切换 DeepSeek / 阿里百炼 / Ollama 三种后端。

4.3 提示词模板示例

合规自查提示词（src/config/prompts.py 中的 COMPLIANCE_CHECK_PROMPT），体现 SDD 规格驱动的设计方法：

```
COMPLIANCE_CHECK_PROMPT = """
```

你是一名海军标准化审查专家。请严格依据提供的现行标准条款，对用户提交的制度/方案文本进行合规性逐条校验。

```
## 校验规则
```

1. 逐条对照：将用户文本与每条相关标准条款逐一比对

- 2. 三态判定：符合（完全一致或严于标准）/ 不符合 / 部分符合
- 3. 引用标注：每条判定必须引用标准编号和具体条款号

```
## 输出格式（JSON）
{
  "total": <int>, "compliant": <int>, "non_compliant": <int>,
  "partial": <int>, "score": <float 0-100>,
  "items": [{ "clause": "...", "result": "...", "reason": "..."} ]
}
```

4.4 Trae CN AI IDE 使用

本项目使用课程指定 AI IDE——Trae CN 进行开发。Trae CN 是字节跳动推出的自适应 AI 原生 IDE，内置智能代码补全、对话式编程助手、代码解释和自动重构功能。

开发过程中使用 Trae CN 完成的主要任务包括：

- 项目骨架搭建：通过 AI 对话生成 FastAPI 应用框架和 Docker 配置；
- 代码重构：将 backend_api.py 拆分为 config / database / document / embeddings / llm / rag / retrieval 模块化结构；
- 提示词优化：迭代调整 14 个 RAG 提示词模板以达到最佳回答质量；
- Bug 修复：通过 AI 辅助定位和修复 SSE 流式响应中断、数据库连接池耗尽等问题。

4.5 安全措施

API Key 通过 .env 环境变量注入，不硬编码在代码中

.gitignore 排除 .env、__pycache__/\、logs/\、data/uploads/

支持 OFFLINE_MODE 完全离线运行（Ollama 本地模型 + 本地嵌入）

LLM 调用前无 API Key 时不发起网络请求

五、测试与评估

5.1 功能测试

测试项	输入	预期结果	状态
智能问答	"舰艇消防系统设计应满足哪些标准？"	返回引用 GJB 条款的回答，标注来源编号	PASS
文本优化	含口语化表达的海军公文	去口语化、统一术语，显示变更统计	PASS
GAP 分析	CB/T 3096-2025 船用窗斗	5 维度诊断报告，识别关联标准缺口	PASS

合规自查	舰艇维修管理制度草案	三态合规报告 + JSON 评分摘要	PASS
文档入库	CB/T 3096-2025.pdf	自动解析、清洗、分块、向量化	PASS
SSE 流式	任意问题	逐 token 实时渲染，无卡顿/中断	PASS
多轮对话	3 轮上下文关联问答	前轮信息正确带入后续回答	PASS
错误处理	未上传文档即提问	友好提示"请先上传相关标准文档"	PASS

5.2 Agent 行为评估

指标	数据	说明
来源引用准确率	87% (13/15)	10 组问题，引用标准编号与原文比对
检索 LRU 缓存命中率	81.2%	512 条目缓存，军事场景重复查询率高
Top-10 召回率	91%	对比 HNSW 检索 vs 暴力余弦检索
回答相关性评分	4.2/5.0	5 位试用者 10 组问题反馈
合规三态准确率	80% (4/5)	对照已知标准文本测试

5.3 Demo 截图



智能问答主界面 — 展示问题输入、SSE 流式回答、引用来源高亮

- 文档上传界面 — 展示 PDF 上传、自动解析进度
- 文本优化界面 — 展示优化前后对比、变更标记
- 标准 GAP 分析界面 — 展示 5 维度诊断报告
- 合规自查界面 — 展示合规评分 JSON 和详细校验结果
- Docker Compose 启动日志 — 展示服务就绪状态

六、系统升级与扩展

6.1 可扩展架构设计

当前架构已预留以下扩展点：

LLM 后端：OpenAI 兼容协议抽象，LLM_BASE_URL 环境变量可切换任意兼容端点

嵌入模型：EMBEDDING_MODEL 环境变量可切换更大规模或英文模型

提示词模板：集中在 config/prompts.py，新增业务场景无需修改代码逻辑

离线模式：llm/offline.py 为 vLLM / llama.cpp 预留接口

6.2 下一阶段计划（v2.1 → v3.0）

升级项	说明	预估周期
LangGraph 多步骤推理	迁移至 LangGraph 状态图，支持 ReAct Agent 循环（思考→检索→反思→再检索→生成）	2 周
MCP 协议集成	将 OCR、文档解析、向量检索封装为 MCP Server 标准工具	1 周
权限分级管理	增加 RBAC 角色（管理员/审查员/查阅者）	1 周
Agentic RAG 升级	自主判断检索充分性、主动追问用户、多跳推理	3 周
离线大模型切换	集成 vLLM，本地部署 qwen2.5:7b/14b	1 周
Benchmark 评估	构建 100+ Q&A 数据集，RAGAS 框架自动评估	2 周

6.3 AI 能力演进路径

短期（v2.1-v2.3）：完善 LangGraph 状态图编排 → 集成 MCP 协议标准化工具 → 新增权限分级管理。

中期（v3.0）：Agentic RAG（自主检索决策+多跳推理）→ Benchmark 评估体系 → 英文标准支持。

长期(v4.0+): 多 Agent 协作(标准审查 Agent+文档撰写 Agent+合规校验 Agent)
→ 自动标准修订建议生成 → 海军标准知识图谱构建。

七、课程总结

7.1 个人收获

技术层面: 完整掌握私有化 RAG 全流程开发, 熟悉 pgvector 向量库、多 LLM 后端兼容、SSE 流式输出与 Docker 容器部署, 能独立搭建行业知识库问答系统。

工程层面: 学会 SDD 规格驱动开发, 先定产品、架构、接口规范再编码, 理解分层解耦、数据本地化等企业级设计原则。

工具层面: 熟练使用 Trae CN AI IDE 完成项目搭建、代码重构与问题调试, 明白 AI 工具是辅助开发、聚焦架构创新。

行业层面: 了解海军 GJB/HJB/CB 标准体系, 掌握军工场景离线、溯源、合规等高安全要求的系统设计思路。

7.2 工程思维转变

本课程最大的收获是跨越了从"代码编写者"到"智能体编排者"的技术鸿沟。在项目初期, 我习惯性地从"写什么代码"开始思考; 经过 SDD 方法论的训练, 我现在更倾向于先定义 Spec (产品规格→架构规格→API 规格), 再考虑实现。这种思维转变体现在:

先设计提示词模板再写代码: 14 个提示词模板的设计过程就是对 AI 行为的规格定义——明确输入、输出格式、约束条件和异常处理;

架构优先而非功能堆砌: 五层架构设计确保了系统的可扩展性, 每层可以独立优化而不影响其他层;

评估驱动改进: 从"感觉效果不好"到"缓存命中率 81.2%, Top-10 召回率 91%"——用数据指导优化方向。

7.3 对课程的建议

建议补充 LangGraph 多步骤推理、RAGAS 量化评估实操教学; 增设 MCP 工具协议、多智能体协作实战内容。